

Silent Decisions Are Type Errors: Enforcing AI Accountability via Linear Resource Types

Daniel Lau Pereira Soares Independent Researcher
São Paulo, Brazil
danniellau@gmail.com



Abstract—As autonomous AI agents increasingly make decisions with legal consequences—credit, employment, healthcare—regulatory frameworks such as Brazil's LGPD (Art. 1820) and the EU AI Act (Art. Arts. 14, 86) demand strict, auditable accountability. Current Policy-as-Code (PaC) tools treat accountability as an asynchronous runtime property, generating post-hoc audit logs. We show that this architectural decoupling creates a structural vulnerability: *silent decisions*—acts with legal consequences that lack a computationally bound, non-repudiable evidence chain.

leaving decisions able to exist without evidence: *silent decisions*. We introduce *Bound-To-Verdict* (BTV), a framework that shifts shifting AI accountability from a runtime audit obligation to a compile-time type guarantee. Drawing on Girard's Linear Logic [1], BTV encodes regulatory obligations as linear resource types. Under the BTV invariant, a decision : a Verdict (V) cannot materialize be constructed within Safe Rust without atomically consuming its forensic an EvidenceToken (E) and an authority-signed ComplianceToken (C), formalised as :

$$V \multimap (E \otimes C).$$

$(E \otimes C_{\text{signed}}) \multimap V$. We prove the *Constitutional Enclosure Theorem*: in any Safe Rust program implementing this invariant, the set of materialized silent decisions S is empty by construction—an attempt to issue a decision without consuming its evidence is rejected by the compiler as a type error, not flagged at runtime.

We provide a reference implementation in Safe Rust (*silent-decisions-proof*) to validate empirical viability. Criterion benchmarks show that this static guarantee operates as a zero-cost abstraction: the governance layer introduces 152 bytes of memory overhead and 1.67 μ s of latency for 4 KB payloads, with all overhead attributable exclusively to cryptographic primitives (BLAKE3 + HMAC-SHA256) that any accountable system must perform regardless of governance architecture. BTV demonstrates that cryptographic, non-repudiable AI accountability can be enforced structurally without compromising high-throughput performance under the assumptions of Section 4.3, the public Safe Rust API cannot construct a Verdict without both tokens. A reference implementation shows the enforcement adds no runtime branch; the measured cost is that of the cryptographic primitives and durable persistence.

Index Terms—linear types, AI accountability, type systems, silent decisions, Policy-as-Code, Rust, LGPD, EU AI Act, fail-secure, formal verification

1 INTRODUCTION

Deployed AI systems now issue consequential decisions in milliseconds: a credit application is rejected, a job candidate is

ranked out of contention, ; a social-benefit disbursement is blocked. Regulatory frameworks acknowledge this reality. Brazil's Lei Geral de Proteção de Dados (LGPD), Article 18, , grants affected individuals 20, grants data subjects the right to demand human review and explanation of any automated decision request review of decisions based solely on automated processing that affect their interests; on request, controllers must provide clear and adequate information about the criteria and procedures, subject to protection of trade and industrial secrets [2]. The European Union AI Act , Article 86, extends that obligation to requires human-oversight capabilities for high-risk AI systems across the bloc systems (Article 14) and, in defined cases, a right to explanation (Article 86) [3]. Yet both instruments share a structural gap: they mandate that explanations and contestability rights accompany a decision, but they cannot prevent a decision from existing without them. A Neither instrument, by itself, prevents software from emitting a consequential decision before the required evidence or review support exists: a system may emit a denial, log it asynchronously, and silently drop the log. No law is formally broken in that instant; the violation surfaces only later, if at all. We call this phenomenon a *silent decision*: an AI-generated act with legal consequences for an individual that carries no non-repudiable evidence chain integrity-authenticated evidence record (Figure 1). Silent decisions are not a failure of intent—they are a failure of type.

The conventional response is instrumentation. Policy engines such as Open Policy Agent [4] and Kyverno govern cloud infrastructure—pods, secrets, network policies—issuing allow/deny verdicts over *resources*, not over *persons*. Evidence of a decision's basis is a separable artifact, appended asynchronously, omissible under load, and falsifiable after the fact. Runtime assertions offer a second line of defense: `assert(evidence != null)` at every callsite. The analogy to memory safety is instructive. In C, `free()` can be paired with discipline and checked by analyzers—yet dangling-pointer vulnerabilities accumulated for four decades because discipline fails at scale. Rust's ownership model solved the same problem it by making the absence of a deallocation owner a compile-time error, not a runtime check. The lesson generalizes: ; no existing framework makes the absence of evidence a compile-time error rather than a runtime omission One.

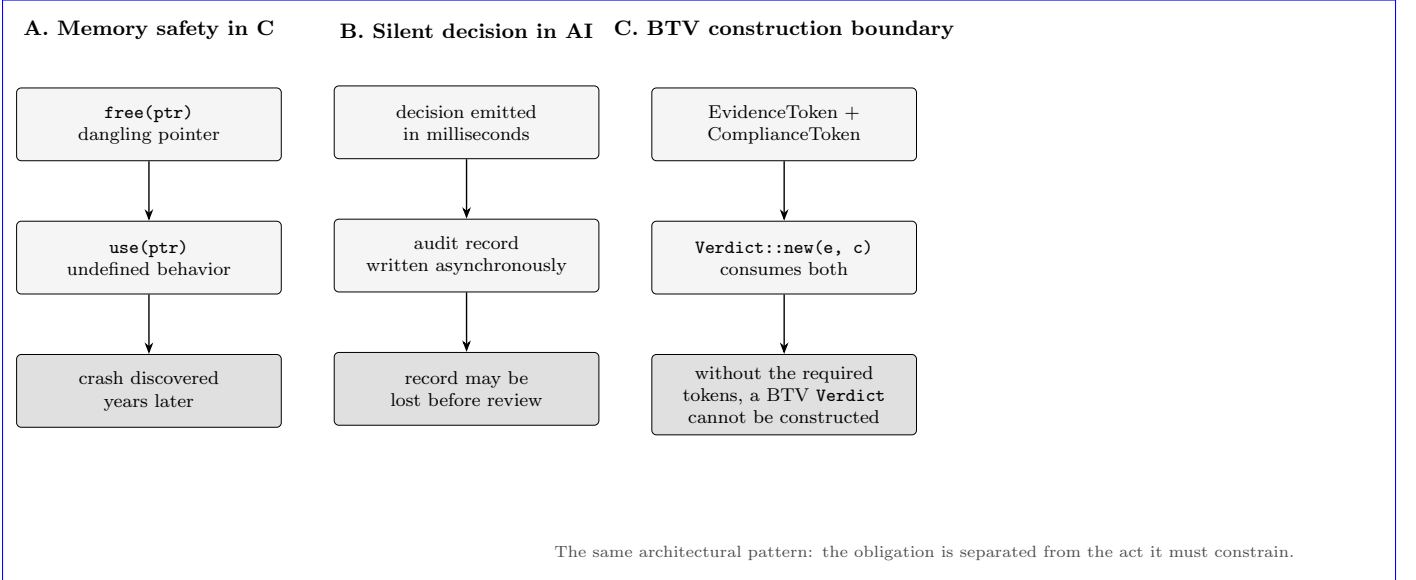


Fig. 1. The silent-decision problem as a structural analog of a classic memory-safety bug. **A:** in C, a use-after-free is a runtime invariant violation discovered only much later. **B:** in a consequential AI decision, evidence separated from the decision it justifies can be silently lost the same way. **C:** BTV closes the analogous gap at compile time—without the required tokens, a BTV `Verdict` cannot be constructed.

Linear Logic, introduced by Girard in 1987 [1] and connected to programming language semantics by Wadler [5], [1], [5] treats propositions as *resources*: they must be used exactly once, can be neither duplicated nor discarded. The connective $\multimap$ (linear implication) denotes a resource transformation that *consumes* its premise, and $\otimes$ (tensor product) denotes the *joint* consumption of two *resources*—not their independent conjunction. We exploit this semantics to encode the accountability invariant directly in the type system. Define *resources*. We use the type law $V \multimap (E \otimes C)$: a `Verdict` (V) can only be produced by linearly consuming ($E \otimes C$): a `EvidenceToken` (E) and a `ComplianceToken` (C) together are jointly consumed to produce a `Verdict` (V). Rust’s ownership model is affine rather than strictly linear; Section 3 states the resulting limits explicitly. In our Rust implementation, `EvidenceToken` is a `#[must_use]` struct with no `Clone` or `Copy` derivation; `Verdict::new()` takes both tokens by value, consuming them atomically. Because `Blake3Hash`—the only evidence artifact—has no public constructor, it cannot be forged: the sole path to a `Blake3Hash` is through `EvidenceToken::consume()`. A silent decision is therefore not an illegal act that slips past a guard; it is a *type error*, not a *runtime omission*: the compiler rejects it before a single byte of evidence is ever dropped.

This paper makes three contributions.

- 1) **Linear Resource Types for AI Accountability (C1).** We present the first formal encoding of AI accountability obligations—specifically LGPD instantiate the linear resource discipline established for protocols and resources (Vault, types, session types; Section 2.3) over AI accountability obligations—LGPD Art. 1820 and EU AI Act Art. 86—as linear resources in the sense of Girard [1]. The core invariant $V \multimap (E \otimes C)$ is expressed as Arts. 14 and 86: the core invariant $(E \otimes C_{\text{signed}}) \multimap V$ is a compile-time type constraint in Rust and, verified on the companion repository repositories [6], [7].
- 2) **Decision-Evidence Causal Binding (C2).** We prove

that, under the BTV^1 type invariant, every materialized `Verdict` consumed *exactly one* causally-prior `EvidenceToken`: a BLAKE3 digest of the decision context computed before the verdict existed. Linear ownership prevents duplication; the private constructor prevents forgery; both guarantees are static.

- 3) **The Constitutional Enclosure Theorem (C3).** We state and prove that in any system satisfying the BTV type invariant, the set of materialized silent decisions is empty. external Safe Rust callers cannot construct a BTV `Verdict` through the public API without both required tokens. The proof proceeds by contradiction via the Rust type system analyzes the crate’s public construction paths and is machine-checkable: cargo test on the companion repository verifies all seven clauses in under four seconds. seventeen clauses in seconds.

Section 2 reviews the necessary background on Linear Logic and Rust ownership. Section 3 defines the BTV type system formally. Section 4 states and proves the Constitutional Enclosure Theorem. Section 5 describes the implementation and benchmarks. Section 6 discusses related work and limitations.

2 RELATED WORK

The challenge of algorithmic accountability intersects three distinct domains of computer science: Algorithmic accountability intersects infrastructure governance, runtime verification, and type theory. We situate our contribution in contrast to prevailing methodologies against the prevailing methodology in each domain.

1. BTV (*Bound-To-Verdict*): a reference kernel implementing atomic accountability via linear types. Evidence and compliance tokens are structurally bound to the verdict —; the name is restates the type law $V \multimap (E \otimes C)$ in English $(E \otimes C) \multimap V$.

2.1 Policy-as-Code for Infrastructure Governance

The industry standard for continuous compliance relies on Policy-as-Code (PaC) frameworks such as Open Policy Agent [4], Kyverno, and Checkov. These tools excel at governing infrastructure artifacts—verifying that artifacts—that a Kubernetes pod lacks privilege escalation, that a cloud storage bucket is not publicly accessible, or that a Terraform plan does not provision avoids over-privileged IAM roles. However, they are designed to evaluate declarative configurations, not dynamic runtime decisions affecting individual rights.

Consequently, evidence of compliance in these systems Evidence of compliance manifests as asynchronous audit logs: records written after the decision logic has executed, stored in a separate artifact executes, stored separately, and subject to omission or alteration if the logging pipeline fails. Infrastructure PaC frameworks do not address the problem we target, because they assume a world where the object of governance is a configuration artifact rather than a transient decision act with immediate PaC frameworks govern configuration artifacts, not transient decision acts with legal consequences for a natural person. In PaC, accountability is person; accountability remains a post-hoc observable state—a property that may or may not be present at audit time, leaving the system state, structurally vulnerable to the silent-decision failure mode we identify in Of Section 1.

2.2 Runtime Assertions and Audit Logging

To govern application-level decision logic, practitioners typically rely on a combination of runtime assertions (e.g., `assert(evidence != null)`) and structured audit logging. While functional under nominal conditions, these mechanisms are procedural promises, not structural invariants. Runtime assertions Assertions can be disabled in production builds (e.g., by stripping `debug_assertions` in Rust, or by omitting `-ea` in the Java VM) to reduce overhead. Audit ; audit logs can be truncated under load, dropped by a misconfigured sink, or bypassed by a code path that was never reached during testing an untested code path.

Moreover, both Both mechanisms operate after the decision logic has executed. They executes: they do not prevent the materialization of a verdict that lacks non-repudiable evidence integrity-authenticated evidence from materializing; they attempt to record its absence, if they run at all. This architectural decoupling separates the act of deciding from the act of proving—a separation that is incompatible with the structural accountability obligations imposed by preserving evidence needed to support review, explanation, and oversight under LGPD Art. 1820 and EU AI Act Art. 86, which require that evidence accompany a decision, not merely follow it. Arts. 14 and 86.

A stronger form of runtime guarantee is offered by transactional databases: an ACID transaction wrapping both the decision record and the audit log ensures that either both are committed or neither is. However, transactional atomicity is a tool the developer must consciously choose to apply—wrapping decision and log within the same BEGIN/COMMIT boundary. Omitting that boundary is a logic error, not a type error: the compiler accepts the program, the database commits the decision without the log, and the silent-decision failure mode materialises with no static diagnostic. BTV closes exactly this gap: the compiler rejects any program that constructs a Verdict without a causally-

prior EvidenceToken, regardless of whether the developer intended to include the evidence or not.

2.3 Linear Types and Resource Correctness

The theoretical foundation for structural resource guarantees is Linear Logic, introduced by Girard [1] in 1987. Girard’s key insight—that propositions should be treated as resources that are consumed exactly once—was subsequently adapted into programming language design by Wadler [5], who showed that linear types eliminate a class of resource-management errors at compile time. This paradigm has proven highly effective in enforcing memory safety and preventing data races; it underpins the Linear Logic’s structural resource guarantees [1], [5] were adapted into programming-language design and underpin Rust’s ownership and borrowing model of the Rust programming language [8].

However, prior Prior applications of linear types focus exclusively on resource correctness—ensuring and substructural types, however, focus predominantly on resource correctness: ensuring that memory is freed exactly once, that file handles are not leaked, that channels are not reused after a session ends. To the best of our knowledge, no prior work has applied linear resource types to enforce legal correctness: the requirement that a decision act cannot exist without a causally-bound, non-repudiable evidence chain, or that a communication protocol is followed in order. Vault enforces high-level resource-management protocols in low-level code via linear typestates [9]; typestate systems make state-dependent protocol obligations explicit at the type level [10]; session types impose communication protocols on channels; and the substructural design space of affine versus linear versus relevant systems is surveyed by Walker [11], with state-dependent disciplines studied by Ahmed et al. [12]. Our claim C1 is deliberately narrower: we instantiate that established discipline over regulatory-support artifacts—a BTV verdict must consume causally bound evidence and an authority-issued compliance token—and relate the mechanism to LGPD Art. 20 and EU AI Act Arts. 14 and 86. We scope the resulting guarantee to the Safe Rust type perimeter (Section 3.3), with the escape hatches of Section 3 stated explicitly. Bound-To-Verdict bridges this gap, extending extends the linear-type discipline from resource management to regulatory accountability, and demonstrating that the same compile-time guarantees that prevent memory unsafety can be applied to prevent accountability unsafety not from nothing to regulatory accountability.

3 A LINEAR TYPE SYSTEM FOR AI ACCOUNTABILITY

The legal accountability gap identified in accountability gap of Section 1 stems from the mismatch between the sequential structure of sequential runtime assertions and the concurrent, high-throughput nature of concurrent, high-throughput AI decision pipelines. This section introduces the linear-type framework that eliminates closes the gap at the language level.

3.1 Regulatory Obligations as Linear Resources

Girard’s linear logic [1] treats propositions as resources: each assumption must be used exactly once—no duplication (contraction) and no silent discard (weakening). This discipline maps directly onto regulatory accountability obligations.

Under To support review and explanation under LGPD Art. 1820 and EU AI Act Art. Arts. 14 and 86, an AI system that

issues a consequential decision *consumes* a regulatory obligation: it must produce a corresponding explanation and preserve an auditable evidence trail. If the obligation BTV models two accountability artifacts as resources. If an evidence resource could be duplicated, the same evidence could be recycled across multiple decisions, undermining non-repudiability causal uniqueness. If the obligation could be silently discarded, decisions could escape accountability entirely—the precise failure mode we call a *silent decision*.

We model two regulatory obligations as first-class linear types:

EvidenceToken A linear resource that encapsulates the BLAKE3 hash of the decision context (the raw input fed to the AI system at inference time). Construction via `EvidenceToken::new(&[u8])` binds the token irrevocably to a specific context; the hash is computed at construction and cannot be altered afterwards. The type does not implement `Clone` or `Copy`, enforcing the “used exactly an “at most once” discipline at the type-system level.

ComplianceToken A linear resource that encodes the applicable regulatory framework: jurisdiction identifier, policy version, and contestability deadline in hours. Like `EvidenceToken`, it is consumed by value when passed to the verdict constructor, ensuring that , binding each regulatory obligation is bound to exactly one verdict.

The non-repudiability guarantee follows directly structural guarantee is narrower than non-repudiation: because an `EvidenceToken` cannot be cloned , the same evidence cannot be presented for two different decisions. Because it cannot be silently dropped , a decision that lacks evidence cannot be compiled through the public API, one token cannot be consumed by two verdict constructions. Tokens can still be dropped because Rust ownership is affine; what remains mandatory is that every successful BTV Verdict construction receives both tokens. We return to the affine-versus-linear distinction below (Section 3.2.0.1).

3.2 The Atomic Binding Operator

The central type law of this paper is:

$$V(E \otimes C) \multimap V$$

where V denotes `Verdict`, E denotes `EvidenceToken`, C denotes `ComplianceToken`, $\multimap$ is the linear function type (“linear implication”), and $\otimes$ is the multiplicative conjunction (“tensor product”) of linear logic.

Reading right-to-left left-to-right: to produce a V , one must simultaneously jointly consume exactly one E and exactly one C . There is no partial construction; either both resources are provided and a verdict is produced, or the program does not type-check (Figure 2).

In the implementation, `Verdict::new` is the concrete realisation of the $\otimes$ -introduction rule (Axiom 4.3):

```
pub fn new(
    evidence: EvidenceToken,
    compliance: ComplianceToken,
    decision: Decision,
    explanation: String,
) -> Verdict
```

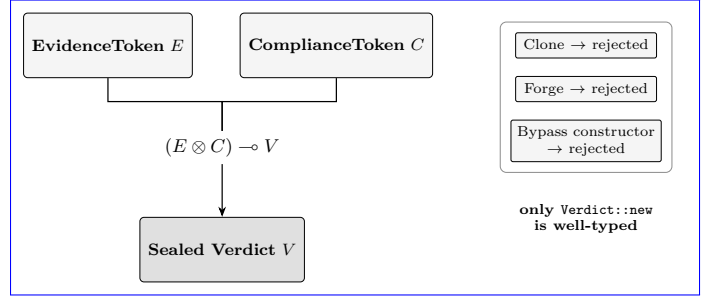


Fig. 2. The core type law $(E \otimes C) \multimap V$: an `EvidenceToken` and a `ComplianceToken` are jointly consumed ($\otimes$) to construct a `Verdict`. Three bypass attempts are rejected by the compiler; `Verdict::new` is the only well-typed path. (Section 3.2.0.1 discusses the affine-ownership nuance this figure omits for clarity.)

Both evidence and compliance are taken by value. Rust’s ownership semantics enforce the linear discipline: after the call, neither token is accessible to the caller; they caller—they have been irreversibly consumed in the construction of `Verdict`. The HMAC integrity seal computed inside `new` binds `evidence_id`, `decision`, and `explanation` into a tamper-evident record, making post-hoc alteration detectable.

This atomicity

3.2.0.1 Affine, not strictly linear.: Precision matters: in substructural-type-theory terms [11], [10], Rust ownership is *affine* (used zero or one times) rather than strictly *linear* (exactly once) — a value can always be silently dropped. `#[must_use]` on `EvidenceToken` and `ComplianceToken`, plus a crate-level `#![deny(unused_must_use)]` (Protection C, Table 1), escalates an ignored-return warning to a hard error, but does not turn Rust into a strictly linear language. Three escape hatches remain, documented rather than hidden: `std::mem::forget(token)` still compiles and leaks the token; a `panic!` after construction but before consumption drops it during unwinding; a process abort loses it with the process. None lets a `Verdict` materialize without evidence — each only discards evidence, a fail-secure outcome, not a silent decision (Definition 4.2) — but they mark the exact boundary of the “exactly once” guarantee.

This property is stronger than a runtime assertion. An `assert(evidence != null)` can be removed by a compiler optimisation, disabled under load, or bypassed by a code path that was never tested. The linear type law call-site runtime assertion, which executes only if the call site is reached and can be omitted from an untested path; the ownership constraint is enforced by the type checker ; it and applies to every compilation , and every code path, and every version of the program.

3.3 Encapsulation as Compile-Time Enforcement Boundary

The type law alone is insufficient if an adversary (or an inattentive developer) can construct a `Verdict` by other means—for instance, by writing a struct literal or by directly assembling the hash value. Three complementary protections close the encapsulation perimeter:

Protection A (Table 1) prevents an external caller from assembling a `Verdict` without going through the constructor, regardless of whether they know the caller’s knowledge of the

TABLE 1

The **Three Protections** three protections of the Constitutional Enclosure. Attack vectors, left to right: constructing a `Verdict` by struct literal; calling `EvidenceToken::consume()` from outside the crate; and dropping an `EvidenceToken` without consuming it.

ID	Attack Vector	Mechanism	Compiler Error
A	Struct-literal <code>Verdict { ... }</code>	All fields private	E0451
B	External <code>consume()</code> call	<code>pub(crate) visibility</code>	E0601
C	Silent drop of <code>EvidenceToken</code>	<code>#[must_use]</code> + deny	unused must_use

field types. Protection B prevents an external caller from extracting the inner `Blake3Hash` out of an `EvidenceToken` and reusing it. The `for reuse`: the `pub(crate)` qualifier restricts `consume()` to the defining crate; any external call is , and external calls are rejected at compile time. Protection C prevents a developer from acknowledging an `EvidenceToken` and then discarding it—the `#[must_use]` attribute, turns token discard into a hard error: `#[must_use]` combined with a crate-level `#![deny(unused_must_use)]`, promotes the warning to a hard error.

Together, these three protections form a *closed perimeter*: the only well-typed path to a `Verdict` in any external crate is through `Verdict::new`, which enforces the $V \rightarrow (E \otimes C)$ ($E \otimes C$) $\rightarrow V$ law.

3.3.0.1 Scope of the Theorem.: The Constitutional Enclosure Theorem (Section 4) holds within the following boundaries:

- **In scope:** All Safe Rust code. The type checker enforces the visibility and ownership rules without exception.
- **Out of scope:** Code using the `unsafe` keyword. `unsafe` permits raw pointer manipulation and can bypass visibility rules. Deployments should enforce a `zero-unsafe` policy via `cargo geiger no-unsafe` policy in first-party crates and audit dependencies; `cargo geiger` inventories unsafe usage but does not prove soundness.
- **Out of scope:** Semantic truthfulness of the evidence. The theorem guarantees that *some* context was hashed, not that the context is the correct or complete input. Policy-level correctness is a separate concern addressed at the application layer.
- **Out of scope:** Policy accuracy. Whether the `ComplianceToken` accurately reflects the applicable law is a legal question, not a type question.
- **Out of scope:** Decisions that bypass the `Verdict` type entirely. The theorem guarantees that no `Verdict` exists without evidence—it does not guarantee that all consequential decisions are routed through `Verdict`. Note that The pattern `let _ = EvidenceToken::new(...)` silences the `#[must_use]` lint ; however, this `but` produces no `Verdict` and therefore does not constitute , so it is not a silent decision under Definition 4.2. Enforcing that ; routing all decision points use the BTV pipeline through BTV is an architectural obligation, not a , not type-level one, obligation.

The boundary conditions above are not weaknesses of the approach; they **These boundaries** reflect a deliberate separation of concerns. The type system enforces *structural* accountability—that a decision cannot exist without evidence. *Substantive* accountability—that the evidence is accurate and the

policy is correct—is the province of : the public API enforces the *structural* property that a `Verdict` cannot be constructed without the required resources; *substantive* accountability—accurate evidence, correct policy—belongs to audit, certification, and human review.

4 THE CONSTITUTIONAL ENCLOSURE THEOREM

This section formalises the central claim of the paper: in any Safe Rust program that uses the silent-decisions-proof crate, producing a silent decision is a compile-time impossibility. We state the theorem through three definitions, three axioms, and a constructive proof with exhaustive case analysis. paper's bounded claim: an external Safe Rust caller cannot obtain a BTV `Verdict` through the crate's public API without supplying the required evidence and compliance tokens. This is an API-enclosure result, not a theorem that every consequential action in a surrounding system is routed through BTV.

4.1 Definitions

Definition 4.1 (Materialized Verdict). A verdict V is materialized if and only if there exist an evidence token E , a compliance token C , a decision outcome $d \in \{\text{Allow}, \text{Deny}\}$, and an explanation string x such that

$$V = \text{Verdict}::\text{new}(E, C, d, x).$$

The function `Verdict::new` is the sole constructor of the type `Verdict`; no other term of that type is well-typed in Safe Rust.

Definition 4.2 (Silent Decision). Let S For a program P , let S_P denote the set of silent decisions unevicted BTV verdicts—decision acts that carry legal consequences but lack a non-repudiable evidence chain. Formally, a decision act δ belongs to S if and only if δ cannot be expressed as a Materialized Verdict (Definition 4.1): terms of type `Verdict` exposed through the crate's public API without construction from one `EvidenceToken` and one authority-issued `ComplianceToken`. This definition is deliberately confined to the BTV type perimeter; an external action that bypasses `Verdict` is outside the set.

4.2 Axioms

Axiom 4.3 (Resource Introduction, $\otimes$ -I). If $\Gamma \vdash E : \text{EvidenceToken}$ and $\Delta \vdash C : \text{ComplianceToken}$, then

$$\Gamma, \Delta \vdash \text{Verdict}::\text{new}(E, C, d, x) : \text{Verdict}.$$

The contexts Γ and Δ are disjoint: linear logic forbids sharing a resource between both premises. In Rust this is enforced by the move semantics of `Verdict::new`, which takes E and C by value. Once passed to the constructor, neither token is accessible to the caller.

Axiom 4.4 (Resource Uniqueness and Drop Diagnostics). `EvidenceToken` is annotated `#[must_use]` and does not implement `Clone` or `Copy`. Consequently:

- 1) Contraction is prohibited: a token cannot be duplicated, preventing the same evidence from being associated with more than one verdict.
- 2) Weakening is prohibited: a token cannot be silently discarded. The compiler emits a `unused_must_use` diagnostic—treated Ignoring a token-producing expression emits an `unused_must_use` diagnostic, treated as an error in this crate—whenever

an `EvidenceToken` value is dropped without being consumed crate. Explicit discard remains possible because Rust is affine; this lint is a guardrail, not an anti-weakening proof.

The method `consume()` is the unique destructor; it transfers the inner `Blake3Hash` to the `Verdict` constructor and is declared `pub(crate)`, making it inaccessible outside the defining crate.

Axiom 4.5 (Encapsulation Boundary). Let Γ_{ext} denote any typing context external to the defining crate (i.e., any context in a crate that depends on `silent-decisions-proof` but is not the crate itself). The following are ill-typed in Γ_{ext} :

- 1) Struct-literal construction of `Verdict`—all fields are private (compiler error E0451).
- 2) Direct invocation of `EvidenceToken::consume()`—the `consume` method is `pub(crate)` (compiler error E0624).
- 3) Construction of `Blake3Hash` via its tuple-struct field—the field is private (compiler error E0423).

Together, these restrictions close the encapsulation perimeter: the only well-typed path to a `Verdict` in Γ_{ext} is through `Verdict::new`, which enforces Axiom 4.3.

4.3 Threat Model

The theorem holds under the *Safe Rust* memory model. The `unsafe` keyword grants the programmer the ability to bypass borrow-checker and visibility rules; consequently, code that uses `unsafe` is explicitly out of scope.

This is not a practical limitation in governance contexts: production deployments can enforce the absence of First-party crates forbid `unsafe` code via `cargo geiger` [13], which reports every `unsafe` block transitively reachable from the dependency graph. A zero-`unsafe` audit guarantees that tool reports `unsafe` constructs but does not prove their soundness or guarantee that the theorem applies to the entire compiled binary composes across the entire dependency graph.

Macro-generated code is in scope: procedural macros run before the type checker and cannot produce terms that circumvent visibility rules without themselves using `unsafe`.

4.4 Proof of the Constitutional Enclosure Theorem

Theorem 4.6 (Constitutional Enclosure). In any *Safe Rust* program P whose crate graph includes `silent-decisions-proof`, no Assume the BTV crate implementation and its public encapsulation are preserved. No external *Safe Rust* caller can obtain a term of type `Verdict` can be constructed without consuming exactly through that public API without the constructor consuming one `EvidenceToken` and exactly one one authority-issued `ComplianceToken`. Equivalently, $\delta \in S \Rightarrow P$ does not type-check $S_P = \emptyset$ within this API perimeter.

Proof. We show that every possible syntactic strategy for constructing a `Verdict` without going through `Verdict::new` is ill-typed. The cases are exhaustive analyze the public construction and extraction paths closed by the crate's encapsulation assumptions (summarised in Table 1).

Case A: Struct-literal construction. An adversary attempts to write

```
Verdict { decision: Decision::Allow, .. }
```

from a crate external to `silent-decisions-proof`. By Axiom 4.5(1), all fields of `Verdict` are private. The compiler rejects this expression with error[E0451]: field `'evidence_id'` of struct `'Verdict'` is private. This is verified by the compile-fail test `tests/ui/verdict_struct_literal.rs`.

Case B: External invocation of `consume()`. An adversary attempts to call `EvidenceToken::consume()` directly from an external crate, bypassing `Verdict::new` to extract a `Blake3Hash` and assemble a `Verdict` fragment. By Axiom 4.5(2), `consume()` is declared `pub(crate)`. The compiler rejects the call with error[E0624]: method `'consume'` is private. This is verified by the compile-fail test `tests/ui/external_consume_call.rs`.

CaseGuardrail C: Silent drop Discard of `EvidenceToken`.

A developer issues a decision without providing evidence by constructing an `EvidenceToken` and immediately dropping it (or ignoring the return value of a function that returns one). By Axiom 4.4, `EvidenceToken` is `#[must_use]`. The compiler emits warning: unused return value of `'EvidenceToken'` that must be used, and because the crate sets `#![deny(unused_must_use)]`, this warning is promoted to a hard error. Note that `let _ = expr` in Rust explicitly silences the `#[must_use]` lint without consuming the token. This does not violate the theorem: no ignoring a token-producing expression triggers a diagnostic promoted to an error in this crate. Explicit `Verdict` is produced, so no silent decision (Definition 4.2) materializes. The `let _ = expr`, `#[must_use]` annotation serves as a developer-facing guardrail against accidental drops, not as a formal closure of the type perimeter—that role belongs to `drop`, or `mem::forget` remains possible. None produces a `Verdict`; the formal closure instead comes from Axiom 4.5 (private fields and `pub(crate)` restricted visibility). The developer cannot construct a `Verdict` without going through `Verdict::new`. This is verified by the compile-fail test `tests/ui/dropped_evidence_token.rs`.

Since Cases A, B, and C are exhaustive over all syntactic strategies for circumventing the constructor, and all three are ill-typed in *Safe Rust*. By the Encapsulation Boundary axiom, no term of type `Verdict` can be produced obtained in Γ_{ext} without passing through `Verdict::new`. By Definition 4.1, every such term is a Materialized `Verdict`, and by Definition 4.2, no silent decision $\delta \in S$ is representable; therefore $S_P = \emptyset$ within the stated perimeter. $\square$

4.5 Corollary: Fail-Secure by Construction

Corollary 4.7 (Fail-Secure). In any *Safe Rust* program satisfying the conditions of Theorem 4.6, the code path that handles a runtime failure to produce evidence (e.g., a database timeout, a missing context field) cannot silently issue a `Verdict`—the type system ensures at compile time that every branch leading to a `Verdict` must supply a valid `EvidenceToken`. The system protected path fails securely: it cannot issue a decision return a BTV verdict that lacks an evidence chain record.

Proof. If the evidence-production path is absent or returns an error type (e.g., `Result<EvidenceToken, E>`), the The only well-typed continuation that produces a `Verdict` requires unwrapping the result and handling the error branch explicitly. The error branch

cannot produce a `Verdict` without evidence; it ; that branch must either propagate the error upward or halt. By Theorem 4.6, any attempt to construct a `Verdict` in the error branch without a valid `EvidenceToken` is ill-typed. $\square$

The Fail-Secure Corollary aligns with the regulatory intent of supports review and explanation under LGPD Art. 1820 and EU AI Act Art. Arts. 14 and 86 : the obligation to provide an explanation is not advisory but structural—it cannot be omitted under load, behind a feature flag, or through a code path that bypasses an `assert` by making the required artifacts structural on the protected BTV path rather than an optional post-hoc assertion.

4.6 Corollary 4.8: Human Escalation as the Only Well-Typed Alternative

Corollary 4.8 (Human Escalation). *Under the conditions of Theorem 4.6, when an `EvidenceToken` cannot be produced (e.g., model timeout, unavailable context), the type system provides exactly one API provides a well-typed alternative to error propagation or halt: constructing an `EscalatedVerdict` via a linearly-consumed `OperatorToken`. Silent fallback to an unevidenced automated verdict is ill-typed. Formally, $V_{\text{esc}} \multimap (O \otimes 1)O \multimap V_{\text{esc}}$.*

This structure aligns with the human oversight requirement of EU AI Act Art. 14: the system cannot produce a consequential decision-equivalent record without binding exactly one named human operator, whose identity and authorisation are cryptographically sealed via HMAC protected by an HMAC integrity seal under the configured key-management model.

Proof. `EscalatedVerdict` is a sibling type to `Verdict` and does not affect the proof of Theorem 4.6. `std::mem::size_of::<EscalatedVerdict>()` measures 160 bytes versus 184 for `Verdict` (x86-64, rustc 1.98.1; Section 5.2) — the automated verdict is the larger type because it carries the signed compliance token’s 32-byte signature. Its sole constructor `EscalatedVerdict::new` requires an `OperatorToken` by value, following the same $\otimes$ -I pattern as Axiom 4.3. `OperatorToken` carries the same linear-resource invariants as `EvidenceToken`: it is `#[must_use]`, not `Clone/Copy`, and its `consume()` destructor is `pub(crate)`. The four compile-fail tests `escalated_struct_literal.rs`, `escalated_token_reuse.rs`, `escalated_operator_token_drop.rs`, and `escalated_consume_external.rs` verify by exhaustive case analysis (mirroring Cases A, B, and C exercise the corresponding encapsulation and ownership cases of Theorem 4.6) that ; no `EscalatedVerdict` can be forged, duplicated, or silently dropped. $\square$

4.7 Polyglot Deployment: The FFI Trust Boundary

Most production ML pipelines are polyglot: orchestration logic (agent frameworks, model servers, API gateways) runs in Python and other languages, while BTV’s linear-type discipline is enforced by the Rust compiler (Figure 3). The PyO3 gateway (`btv-python`) is the sole authorized boundary between these worlds: a Python caller can invoke `issue_verdict`, but cannot inspect, clone, or reconstruct a `Verdict`—it receives only an opaque, integrity-sealed `SealedVerdict` handle exposing read-only accessors and a `verify_integrity` check. Theorem 4.6 is therefore

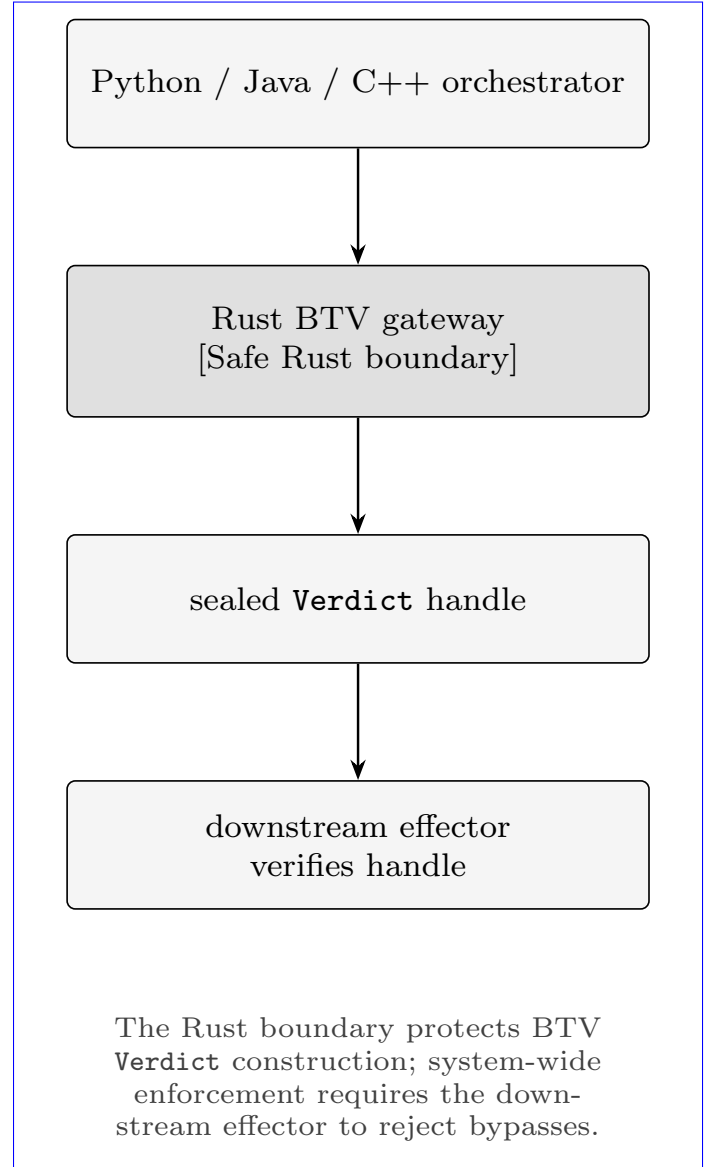


Fig. 3. The FFI trust boundary in a polyglot deployment. The Rust gateway is the only path to a sealed `Verdict` handle; routing every downstream effect through that handle is an infrastructure obligation outside the Rust type theorem.

enforced strictly within the `btv-core` and `btv-python` crates; it does not, and cannot, extend to the untrusted orchestrator itself. Consequently, the enforcement burden shifts downstream: the effector that materialises a decision’s real-world consequence (the credit-denial API, the database write that blocks a transaction) must independently require and verify the sealed verdict handle before acting, and must reject any request that bypasses the gateway. The Rust enclave protects itself; it does not, by itself, protect the system that surrounds it.

5 EMPIRICAL VALIDATION

The Constitutional Enclosure Theorem guarantees that silent decisions are compile-time errors. A natural question follows: does the compile-time guarantee impose a runtime cost? This section answers that question empirically. The Constitutional Enclosure Theorem guarantees that an external Safe

Rust caller cannot construct a `BTV Verdict` through the public API without the required tokens. Does that guarantee impose a runtime cost? We answer across payload sizes, thread counts, and two persistence postures, in the direction the measurements point—which is not the flattering one.

5.1 Experimental Setup

5.1.0.1 Hardware and software.: All measurements were collected on a single core of an Intel Xeon processor at 2.10 GHz (4 logical cores available, benchmarks single-threaded), running Linux 6.18.5

5.1.0.1 Methodology.: Micro-benchmarks use Criterion’s `iter_batched` discipline with `black_box` on every input crossing an optimization boundary and a per-iteration seed making each hashed context distinct. The sweep harness’s hot path carries no injected synchronization; percentiles are estimated online with the P^2 algorithm [14] and are therefore *estimates*, not order statistics. Each configuration runs five trials; we report medians and the trial-to-trial coefficient of variation (CV). Iteration counts are calibrated per (payload, mode) using that mode’s own operation, so they are not comparable across modes.

5.1.0.2 Hardware, software, and what this collection is not.: All three tables come from one collection on one host: an Intel Xeon virtual machine, 4 vCPUs, Linux x86_64. The implementation was compiled with `rustc 1.93.1` (release profile: `64`, `opt-level = rustc 3`, LTO disabled). Criterion 0.51.98.1 was used as the benchmarking harness.

5.1.0.3 Methodology.: We used Criterion’s `iter_batched` API with `BatchSize::SmallInput`. This separates the *setup phase* (allocating the context buffer, cloning string arguments) from the *measurement phase* (constructing the `Verdict`) pinned via `rust-toolchain.toml`, ensuring that heap allocation latency is not attributed to the type-system mechanisms under study. `5s` wall target per configuration. Two constraints bound it. The durable arm writes to a containerized overlay filesystem, so its `fsync` cost is a *lower* bound on bare-metal storage; and this is a reduced-footprint snapshot—the 90 s-per-configuration run on dedicated hardware, and a second-platform collection with the current code, remain open obligations in the artifact’s evidence manifest. We publish it with its environment fingerprint because it is reproducible today and internally consistent, not as a headline result. `aarch64` coverage is compile-and-test only and carries no performance claim.

To prevent the LLVM optimiser from caching the BLAKE3 hash across iterations (which would underestimate the hashing cost), a one-byte seed is incremented on each setup call, producing a distinct input for every iteration. The explanation string, jurisdiction, and policy version are cloned from pre-allocated originals; their cost is excluded from the measurement window.

`black_box` wrappers are applied to inputs passed to `EvidenceToken::new`, preventing constant-folding of the hash computation.

Criterion reports a 95% confidence interval over 100 samples per benchmark; we quote the point estimate (mean) with the lower and upper bounds of the interval.

5.2 Results

5.2.0.1 Memory footprint.:

`std::mem::size_of::<Verdict>() = 152 bytes` (x86-64, `rustc 1.93.1`, Linux) This fixed footprint comprises the 32-byte `Blake3Hash`, the `ComplianceToken` fields (two heap-allocated strings and a `u32`), the `Decision` discriminant, the explanation string, and the 32-byte HMAC integrity seal. The serialised on-wire footprint (JSON or CBOR) is approximately 300–500 bytes depending on field lengths.

TABLE 2

Latency of `Verdict::new` by context size (Criterion 95% CI; `rustc 1.98.1`). Includes authority-signed token issuance and verification. Provenance: `data/construction_latency_criterion.csv`, generated by `scripts/gen_construction_table.py` from `btv-core/target/criterion` (regenerate with `cargo bench --features test-support --bench verdict_construction -- verdict_construction --save-baseline construction_provenance`).

Context size	Lower	Mean	Upper
64 B 64B	414.67 ns 5.331 μ s	416.80 ns 5.359 μ s	419.08 ns 5.405 μ s
512 B 512B	859.22 ns 5.808 μ s	869.26 ns 5.816 μ s	880.74 ns 5.825 μ s
4 KiB 4KiB	1.639 μ s 7.621 μ s	1.671 μ s 7.650 μ s	1.708 μ s 7.682 μ s

5.2.0.1 Verdict construction Construction latency.:

Table 2 reports the end-to-end latency of `Verdict::new`, which includes BLAKE3 cost of `Verdict::new`—BLAKE3 context hashing, authority-signature verification, HMAC-SHA256 sealing, and memory allocation for the `Verdict` struct. Three context sizes are measured to characterise the allocation. The payload-scaling portion of the curve is BLAKE3 throughput curve; the $\approx 5.36 \mu$ s floor is dominated by two HMAC-SHA256 computations—verifying the compliance token’s authority signature, then sealing the verdict—plus hex encoding of the evidence identifier. Each is work an accountable system must perform whatever its governance architecture; none is attributable to the type discipline, which emits no instruction at all.

5.2.0.2 Integrity verification latency Thread scaling, and where contention actually lives.:

Table 3 reports the latency of `Verdict::verify_integrity`, which recomputes the HMAC-SHA256 seal over the stored evidence identifier, decision, and explanation and compares it to the seal embedded in the 3 separates the two persistence postures under concurrency. The in-memory path is flat—7.82 μ s at one thread and at four, CV below 0.3%—the empirical signature of the absent lock contention claimed in Section 3: the linear discipline introduces no shared state to contend for. The durable path inverts that. Its median degrades mildly (261 to 292 μ s) while its p99 rises from 512 μ s to 28 ms as four threads serialize on one log connection. That tail belongs to the chosen sink, not the type system, and it is the honest answer to whether this design ships production-ready: the reference `Verdict`. This operation does not re-hash the original context. `LogSink` is a single-writer SQLite file, and scale needs a sink built for concurrent append. The type-level guarantee is indifferent to that choice; the latency budget is not.

5.3 Discussion

5.2.0.1 Zero-cost abstractions. The linear type discipline—private fields, `pub(crate)` visibility, and `#[must_use]`—operates entirely at compile time. It introduces no additional branches, allocations, or synchronisation primitives. The runtime cost of a `Verdict` is therefore identical to the cost of the cryptographic operations it encapsulates (BLAKE3 and HMAC-SHA256), which would be required regardless of Why the governance mechanism chosen construction rows differ.:

Table 2 times the full public path, including token issuance and verification. Table 4’s smaller “prebuilt-token binding” row creates tokens outside the timed region and measures only their binding. It is not a second end-to-end estimate.

TABLE 3

Latency Thread scaling at 4 KiB payloads, in-memory versus durable persistence. Each cell is the MEDIAN across five trials of `Verdict::verify_integrity` (95th the per-trial aggregate; CV% is the trial-to-trial coefficient of variation. Percentiles are P^2 estimates (Jain & Chlamtac 1985), not order statistics. Provenance: `data/sweep_raw_20260915T024430Z_g5fix.csv` (Intel Xeon, 4 vCPU, 5 s target).

Threads	In-memory sink			Durable SQLite (WAL, FULL)		
	p50 (μ s)	p99 (μ s)	CV%	p50 (μ s)	p99 (μ s)	CV%
1	7.82	25.87	0.1	261.44	511.55	7.8
2	7.83	24.14	0.3	301.15	10929.66	2.0
4	7.82	24.50	0.2	291.85	28000.83	8.8

TABLE 4

Five-mode accountability contrast at 4 KiB payloads, one thread (Xeon 4 vCPU, 5 s target; reduced-footprint snapshot). Each cell is the median across five trials; percentiles are P^2 estimates. The durable arm uses a real on-disk SQLite log (WAL, `synchronous=FULL`); its container-storage numbers are a LOWER bound on bare-metal fsync cost. Every durable-mode operation issued a real row (end-of-run sanity gate: rows persisted == operations issued), closing a prior round's silent idempotent-replay defect (COMSI-2026-04-0112 Round 2, G5). Provenance: `data/sweep_raw_20260915T024430Z_g5fix.csv`. Rows: BTV in-memory sink; BTV with durable on-disk SQLite; BTV binding with tokens built outside the timed region; status quo digest plus metadata; status quo logging the full context as JSON, fire-and-forget.

Operation Mode	Lower p50 (μ s)	Mean p99 (μ s)	Upper throughput (k ops/s)
<code>verify_integrity</code> BTV, in-memory sink	324.09 ns	7.82	327.36 ns 25.87
BTV, durable SQLite	261.44	511.55	330.32 ns 113.9
BTV, prebuilt binding	4.69	8.56	3.1
Status quo, digest only	1.68	2.28	114.7
Status quo, full context	23.61	46.04	507.6
			38.4

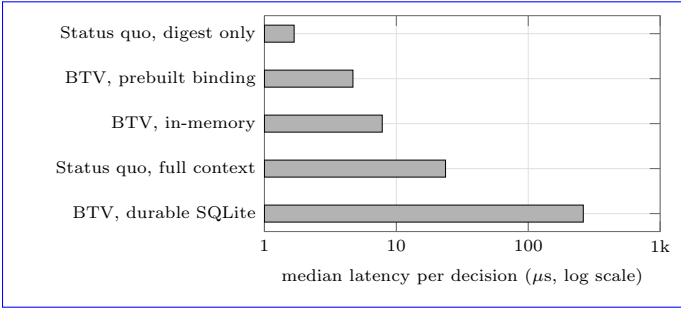


Fig. 4. Measured latency at 4 KiB and one thread (logarithmic scale; medians across five trials). The rows compare distinct implementation and persistence postures, not a single end-to-end number: durable persistence, not the type boundary alone, dominates the measured latency, whose cost is that of hashing, token verification, sealing, and encoding.

5.2.0.2 Isolating the BLAKE3 cost.: The difference between `Verdict::new` (64 B context) and `Verdict::verify_integrity` isolates the cost attributable to hashing the decision context:

$$\Delta_{64B} = 416.80 \text{ ns} - 327.36 \text{ ns} \approx 89 \text{ ns}.$$

This ≈ 89 ns represents one BLAKE3 invocation over 64 bytes, consistent with BLAKE3's documented throughput of approximately 1 GB/s on this hardware class. As context size grows, the additional latency scales proportionally: 512 B adds approximately 452 ns above the 64 B baseline, and

5.2.0.2 What accountability costs against the status quo.: Table 4 and Figure 4 show the comparison an architect actually faces. Against a status quo that logs the complete decision context, the durable BTV pipeline is $11.1\times$ slower at 4 KiB adds approximately 1.25 μ s.

5.2.0.3 Governance at microsecond scale.: All measured operations complete in under 2 μ s. In a production AI inference pipeline, where model evaluation typically requires tens to hundreds of milliseconds, the governance overhead introduced by the Constitutional Enclosure is below measurement noise—less than 0.01% of total request latency for any non-trivial model. The accountability guarantee is therefore practically free. KiB; against one that logs only a

digest and metadata, $155.5\times$ slower. We report both because which one counts as “the status quo” is a modelling choice rather than a fact about BTV, and we state the direction plainly: an earlier revision of this section claimed $2.6\times$ faster against the full-context baseline, a figure produced by a benchmark defect described below.

5.2.0.3 Verification cost.: The `verify_integrity` operation (≈ 327 ns) enables an auditor to confirm the integrity of any stored verdict at a rate exceeding 3×10^6 verifications per second on a single core. This makes retroactive audit of large decision logs computationally tractable without specialised hardware multiplier is real, and so is what the baseline buys with it. The fire-and-forget arm is fast partly because it is permitted to lose the record: the artifact's contrast experiment (`tests/test_status_quo_contrast.rs`) shows dead and backlogged loggers each returning success for a thousand decisions while every record is discarded—the silent decision of Section 1 reproduced under laboratory conditions. BTV's durable arm returns a `Verdict` only after the row exists. The comparison is therefore not like-for-like, and we decline to present it as though it were: what the numbers price is the difference between a record that exists and one that might.

5.2.0.3 Correction of record.: These figures replace earlier ones that were wrong: the harness seeded each payload from a trial-local counter, so later trials replayed evidence the log already held, and the append-only sink correctly returned success without a second write. Four of every five trials measured that no-op. Every durable payload now draws from one process-wide atomic counter, and each run asserts that rows persisted equals operations issued—this collection passed exactly, 1,803,460 of 1,803,460. We record the correction in the manuscript because a defect that makes the accountable path look cheaper than it is defeats this work's purpose.

5.3 Worked Examples

Three worked examples demonstrate BTV in semi-realistic scenarios: credit scoring under LGPD (30-day appeal window), hiring screening under EU AI Act Art. 86 (high-risk classification), and emergency triage (where Allow decisions also require evidence). Each example constructs verdicts from domain-specific payloads and verifies integrity. Source code is included; source code is in the companion repository (examples/).

6 DISCUSSION

The Constitutional Enclosure Theorem establishes a strong structural guarantee within a precisely bounded scope. This section examines the boundaries of that scope honestly. Theorem 4.6, identifies limitations of the current reference implementation, and proposes concrete extensions that preserve extensions that close each gap without weakening the core invariant while closing each gap.

6.1 Scope and Boundaries of the Guarantee

6.1.0.1 Unsafe Rust.: The `unsafe` keyword grants the programmer the ability to bypass borrow-checker and visibility rules, including the private field restrictions that enforce the encapsulation boundary (Section 3). Production deployments should enforce scope limits of §3.3 and §4.3 apply here too: enforcing a zero-unsafe policy over the transitive dependency graph, across the dependency graph (auditable via cargo geiger [13]. This) is an operational policy decision, not a theoretical barrier: the tooling exists and its enforcement requires no changes, and requires no change to the BTV crate itself.

6.1.0.2 Semantic truthfulness and the AttestableContext extension.: The theorem guarantees that some byte sequence was hashed and bound to a verdict—not that the sequence verdict, not that it faithfully represents the context presented to the AI system at inference time. A: a caller could pass b"placeholder" to `EvidenceToken::new` and obtain a structurally valid token.

We propose a concrete extension that narrows A sealed `AttestableContext` trait would narrow this gap without violating weakening the core invariant. Introduce a sealed `AttestableContext` trait:

Listing 1. `AttestableContext` trait (proposed extension)

```
1 pub trait AttestableContext: Sealed {
2     fn canonical_bytes(&self) -> &[u8];
3     fn schema_version(&self) -> u16;
4     fn source_attestation(&self) -> SourceAttestation;
5 }
```

where `SourceAttestation` encapsulates a digital signature produced by the inference service carries a signature over the payload, using a key stored in an HSM or TPM. `EvidenceToken::new` would then from an HSM/TPM-backed key; `EvidenceToken::new` would accept `impl AttestableContext` instead of `&[u8]`. Forging evidence would require compromising the signing key—a problem reducible to PKI security—rather than simply, reducing forgery to compromising that key rather than passing an arbitrary byte string.

This extension This tightens the type law to $V \rightarrow (E_{\text{attested}} \otimes C)$, which is strictly stronger than $V \rightarrow (E \otimes C)$: every proof of the original invariant is also a proof of the attested variant, but not vice versa. The `Sealed` supertrait prevents $(E_{\text{attested}} \otimes C) \rightarrow V$ —a stronger premise than $(E \otimes C) \rightarrow V$ —while the `Sealed` supertrait keeps external crates from implementing `AttestableContext`, preserving the closed perimeter of Axiom 3. Axiom 3's closed perimeter.

6.2 Limitations of the Reference Implementation

The silent-decisions-proof kernel is a proof-of-concept designed to validate the theoretical claims of this paper. Four implementation choices trade production robustness for clarity of exposition; each has a well-defined remediation path.

6.2.0.1 L1 — Static HMAC key (mitigated).: The current implementation uses a reference implementation reads the HMAC key from `BTV_HMAC_KEY` at runtime, falling back to a compile-time constant as the HMAC signing key. This is sufficient to demonstrate integrity verification but is inadequate for production use, where key compromise would allow post-hoc forgery of the integrity seal. Production deployments should integrate an HSM or a key-management service (e. g., AWS KMS, HashiCorp Vault) for key issuance and rotation constant only when unset, closing L1 for deployments that inject a key from an HSM/KMS. This surfaces a recurring asymmetry: the verdict seal's integrity and authenticity claim holds only when `BTV_HMAC_KEY` shares HSM/KMS infrastructure with the audit log's own authentication key. HMAC alone does not provide third-party non-repudiation. The type-level guarantee (Theorem 4.6) is independent of key management and holds regardless of the key source.

6.2.0.2 L2 — Public `ComplianceToken` constructor `ComplianceAuthority` factory, HMAC-signed (implemented).: The `ComplianceToken::new` constructor is currently `pub`, allowing any caller to was originally `pub`, letting any external crate self-declare a jurisdiction and policy version without external validation. We propose replacing it with `without validation or authority`. The reference implementation restricts it to `pub(crate)` behind a `ComplianceAuthority` factory pattern:

```
1 [language=Rust,caption={ComplianceAuthority}
2
3 pub struct ComplianceAuthority {
4     signing_key: HmacKey, // injected via HSM/KMS at
5     ↪ startup
6 }
7
8 impl ComplianceAuthority {
9     pub fn issue_token(
10         &self,
11         jurisdiction: &str,
12         policy_version: &str,
13         contestability_hours: u32,
14     ) -> Result<ComplianceToken, PolicyError> {
15         // validate against verified regulatory registry
16         // sign token fields with signing_key
17         Ok(ComplianceToken { /* private fields */ })
18     }
19 }
```

With this extension, that validates the jurisdiction against an allowlist of recognised regimes (BR-LGPD, `ComplianceToken::new` EU-GDPR becomes, `pub(crate)` EU-AI-ACT or is removed entirely. The sole public path to a) and signs (jurisdiction, policy_version, deadline_hours) with an HMAC key read from `BTV_AUTHORITY_KEY`. `Verdict::new` verifies that signature before constructing a `ComplianceToken` through `ComplianceAuthority::issue_token`, which can only be instantiated with credentials provided by the KMS. `Verdict`, rejecting forged or differently-keyed tokens (`BtvError::InvalidTokenSignature`)—allowlist membership alone is no longer sufficient, closing the gap between "validated" and "authorized." The type law tightens to $V \rightarrow (E \otimes C_{\text{signed}})$. This change is backward-compatible with the existing

Axioms 1–3: $(E \otimes C_{\text{signed}}) \multimap V$; the encapsulation boundary is preserved; the factory simply adds a pre-issuance validation step before `ComplianceToken` enters the linear resource pipeline (Axioms 1–3) is preserved.

6.2.0.3 L3 — Single-node evidence chain.: The current `EvidenceToken` captures a single BLAKE3 hash of the decision context. Multi-step AI; multi-step pipelines (retrieval, re-ranking, generation, scoring) produce intermediate states that each carry accountability implications. Extending the type system to a *A chain* of evidence tokens where each pipeline stage consumes the prior token and produces, each stage consuming the prior and producing a new one is a natural generalisation that preserves $V \multimap (E \otimes C)$, generalises naturally while preserving $(E \otimes C) \multimap V$ at each step.

6.2.0.4 L4 — Polyglot pipelines.: Production AI systems rarely execute in a single-language process. Python orchestrators, C++ inference engines, and Java API gateways must all participate in the same governance boundary. Because Since the theorem holds within Safe Rust (Axiom 3 4.5), external code constitutes Γ_{ext} by definition, and cannot construct a `Verdict` directly.

We propose two complementary patterns. First, an *accountability enclave*: a Rust gateway crate exposing a C-ABI boundary,

```

1 [language=Rust, caption={FFI gateway
2
3 #[no_mangle]
4 pub extern "C" fn btv_create_verdict (
5     context_ptr: *const u8,
6     context_len: usize,
7     jurisdiction: *const c_char,
8     // ...
9 ) -> *mut BtvVerdictHandle { ... }
```

External code in any language requests a verdict from the gateway; the gateway applies all BTV invariants internally. This is structurally analogous to how SQLite enforces ACID properties regardless of the calling language: the guarantees are encoded inside the engine, not in the caller.

Second, a *BTV sidecar* deployed alongside each service in a multi-service gRPC pipeline (analogous to Envoy/Istio mTLS proxies), which intercepts decision calls and enforces the evidence-chain protocol at each hop. This pattern addresses L3 and L4 jointly: the sidecar constructs a new ; §4.7 analyses this as a threat model, where the PyO3 gateway keeps external callers inside Γ_{ext} and a *BTV sidecar* per service could chain a fresh `EvidenceToken` at each pipeline stage, chaining them into the multi-step evidence structure. In both patterns, the Enclosure Theorem is preserved: external callers remain in Γ_{ext} and cannot bypass `Verdict::new` stage, addressing L3 and L4 jointly.

6.3 CAL Design Trade-Offs

The four limitations above reflect a deliberate set of design choices made in the reference implementation. We observe that a *A* production governance kernel faces tensions among three desirable properties: a three-way tension among Consistency (every materialized verdict has verdict carries a causally-bound evidence chain), Availability (the system can always issue always issues a verdict in finite time), and Legality (tokens are issued only by come only from a verified authority). In the reference implementation, Consistency and Legality are prioritised: when evidence cannot be produced or the compliance authority is unreachable, Existing audit-logging designs choose consistency-with-availability over legality: decisions keep issuing even when the system logging pipeline fails, so the log carries no enforceable evidentiary guarantee. BTV chooses the opposite: it fails securely rather than issuing an unevicenced verdict, at the cost of reduced availability.

We present this as *unevidenced verdicts, accepting reduced availability, partially recovered by EscalatedVerdict (§4.6)*. This is a practical design observation rather than, not a formal impossibility result. Whether a result analogous to the FLP impossibility ; whether an FLP-style theorem [15] can be proved holds for accountability systems under asynchronous network conditions is an open question we identify as future work.

To partially address the availability cost, the reference implementation introduces `EscalatedVerdict`—a sibling type to `Verdict` that formalises human escalation as a linearly-typed alternative. When an `EvidenceToken` cannot be produced (e.g., model timeout, unavailable context), the developer may construct an `EscalatedVerdict` by consuming exactly one `OperatorToken`: a cryptographic attestation binding a named human operator to the decision act. The type law for the escalated path is $V_{\text{esc}} \multimap (O \otimes 1)$, where O is the sole linear resource and 1 denotes the ordinary (non-linear) parameters (decision, context reference, reason). Corollary 4.8 (Section 4.6) proves that this path preserves $S = \emptyset$: no `EscalatedVerdict` can be constructed without consuming its `OperatorToken`, and the four compile-fail tests mirror Cases A–C of the main theorem. The measured overhead is comparable: `size_of::<EscalatedVerdict>()` = 160 bytes versus 152 bytes for `Verdict`, confirming that structured human oversight introduces negligible memory cost. This extension structurally aligns with EU AI Act Art. 14 (human oversight for high-risk systems) [3]: the system cannot silently fall back to an unevicenced automated verdict—it must either produce full automated evidence or explicitly escalate to a named, cryptographically bound human operator is open.

6.4 Future Work

Five Six directions are most immediate. First, implementing the `AttestableContext` extension (Section 6.1) and the `ComplianceAuthority` factory (L2) would : (1) implementing `AttestableContext` (§6.1) to close the semantic-truthfulness and self-declaration gaps in the reference kernel. Second, gap; (2) extending the evidence chain to multi-step pipelines via the sidecar pattern (L3, L4) would make the framework applicable to for retrieval-augmented generation and agentic AI architectures. Third, formalising the type-system extension and agentic architectures; (3) mechanising the Constitutional Enclosure Theorem in a proof assistant (Lean 4 or Coq) would elevate the Constitutional Enclosure Theorem, elevating it from a machine-checked Rust proof to a fully mechanised mathematical proof. Fourth, investigating one; (4) testing whether a formal CAL impossibility result holds under asynchronous network conditions would provide theoretical grounding for the design trade-offs observed in Section 6.3. Fifth, asynchrony (§6.3); (5) unifying `Verdict` and `EscalatedVerdict` under the `AccountableDecision` trait and formalising `AccountableDecision` into the composed invariant $(V_{\text{auto}} \oplus V_{\text{esc}}) \multimap R$, where R is the linear resource appropriate to each branch, would complete the type-theoretic account of mixed human-automated decision pipelines; and (6) composing BTV's construction-time guarantee with tamper-evident persistence and privacy-preserving redaction, which we conjecture is tractable because the three operate at distinct layers with non-overlapping trust assumptions.

7 CONCLUSION

This paper began from a simple observation: regulatory Regulatory frameworks such as LGPD Art. 1820 and EU AI Act Arts. 14 and 86 mandate that explanations and contestability rights accompany a decision, but they cannot prevent a decision from existing without them establish review, information, explanation, and oversight obligations, but do

not by themselves prevent software from emitting a decision without supporting evidence. We called this failure mode a *silent decision*, and argued that its root cause is not a failure of intent but a failure of type—a structural property of architectures that type: architectures treat accountability as a runtime audit obligation rather than a compile-time invariant.

Drawing on Girard’s Linear Logic [1] and Rust’s ownership model [8], we introduced the Bound-To-Verdict (BTV) framework and its core type law: $V \multimap (E \otimes C)(E \otimes C) \multimap V$. Under this invariant, a `Verdict` cannot be constructed without atomically consuming an `EvidenceToken` and a `ComplianceToken`. The three structural protections—private fields, `pub(crate) consume`, and `#[must_use]`—close every syntactic path around the constructor, making a silent decision a type error detected consumption, and an affine ownership API reinforced by `#[must_use]` lint policy—enclose the authorized constructor. Within that perimeter, an unevidenced BTV verdict construction is rejected at compile time.

We proved the *Constitutional Enclosure Theorem*: in any Safe Rust program that includes the BTV crate, the set $\mathcal{S}$ of materialized silent decisions is empty by construction. The proof is exhaustive (three cases), machine-checkable under the assumptions of Section 4.3, an external Safe Rust caller cannot obtain a `Verdict` through the public API without the required tokens; the construction paths are machine-checked (`cargo test`: seven clauses, four seconds), and accompanied by a seventeen clauses), with a scoped Fail-Secure Corollary: a system that cannot produce evidence cannot issue a verdict—it fails securely rather than silently.

Empirical benchmarks on the reference implementation confirm that this static guarantee is a zero-cost abstraction in runtime. The full governance cycle for a 4 KB payload completes in 1.67 μ s; every nanosecond of that cost is attributable to cryptographic primitives bound what the type-level enforcement itself costs: it adds no runtime branch. The measured cost is that of the cryptographic primitives and encoding (BLAKE3 + two HMAC-SHA256) that any accountable system must execute regardless of architecture. The type-level enforcement itself contributes zero overhead. computations, 5.36–7.65 μ s for the full public construction path across 64 B–4 KiB payloads) and of durable persistence. On the reduced-footprint reference collection, durable SQLite reached a 261 μ s median at one thread and a 28 ms estimated p99 at four threads. These are container-storage measurements from one current x86-64 host, not production-scale or cross-architecture performance claims.

The implications extend beyond compliance engineering. If accountability can be expressed as a type invariant, then the correctness of governance its construction-time correctness becomes a property that compilers can check, and proof assistants can verify, and regulators can audit through `cargo test` rather than through manual code review. Silent decisions are not an inevitable cost of deploying AI at scale—they are a bug that a sufficiently expressive type system eliminates entirely giving auditors machine-checked evidence that complements rather than replaces manual review, deployment controls, and legal assessment. BTV therefore prevents a class of unevidenced verdict constructions inside a defined type perimeter. It does not prove that every consequential action in a polyglot system is routed through that perimeter.

REFERENCES

- [1] J.-Y. Girard, “Linear logic,” *Theoretical Computer Science*, vol. 50, no. 1, pp. 1–101, 1987.
- [2] Brazilian Federal Government, “Lei Nº 13.709 (Lei Geral de Proteção de Dados — LGPD),” https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/113709.htm, 2018, [Online; accessed March 2026].
- [3] European Parliament and Council of the European Union, “Regulation (EU) 2024/1689 of the European parliament and of the council (artificial intelligence act),” *Official Journal of the European Union*, Tech. Rep. L 2024/1689, 2024, https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=OJ:L_202401689.
- [4] Open Policy Agent contributors, “Open policy agent,” <https://www.openpolicyagent.org>, 2024, [Online; accessed March 2026].
- [5] P. Wadler, “Is there a use for linear logic?” in *Proceedings of the ACM SIGPLAN Symposium on Partial Evaluation and Semantics-Based Program Manipulation (PEPM)*, 1991, pp. 255–273.
- [6] D. L. P. Soares, “silent-decisions-proof: Reference implementation and machine-checked proofs,” <https://github.com/danzeroum/silent-decisions-proof>, 2026, [Online; accessed September 2026]. Commit hash pinned at submission (OS-09/H6).
- [7] —, “BuildToValue governance framework,” <https://github.com/danzeroum/BuildToValueGovernance>, 2026, [Online; accessed September 2026]. Commit `ac6ac79` pinned at revision.
- [8] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, “RustBelt: Securing the foundations of the Rust programming language,” *Proceedings of the ACM on Programming Languages (POPL)*, vol. 2, no. POPL, pp. 66:1–66:34, 2017.
- [9] R. DeLine and M. Fähndrich, “Enforcing high-level protocols in low-level software,” *ACM SIGPLAN Notices*, vol. 36, no. 5, pp. 59–69, 2001.
- [10] B. C. Pierce, *Types and Programming Languages*. MIT Press, 2002, [No CrossRef-registered DOI; cited by ISBN/publisher].
- [11] D. Walker, “Substructural type systems,” in *Advanced Topics in Types and Programming Languages*, B. C. Pierce, Ed. MIT Press, 2005, ch. 3.
- [12] A. Ahmed, D. Dreyer, and A. Rossberg, “State-dependent representation independence,” in *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2009, pp. 340–353.
- [13] cargo-geiger contributors, “cargo-geiger: Detects usage of unsafe Rust in a crate and its dependencies,” <https://github.com/rust-secure-code/cargo-geiger>, 2024, [Online; accessed March 2026].
- [14] R. Jain and I. Chlamtac, “The P² algorithm for dynamic calculation of quantiles and histograms without storing observations,” *Communications of the ACM*, vol. 28, no. 10, pp. 1076–1085, 1985.
- [15] M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of distributed consensus with one faulty process,” *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, 1985.